GenVoice Care: CarePal - An AI-Powered Chatbot for Elderly Wellness Monitoring and Caregiver Coordination

Akshaya G[1]
g.akshayasam@gmail.com
B.Tech, Dept of CSE

Sruthi E[2]
Rechealchristy0@gmail.com
B.Tech, Dept of CSE

Archana V[3]
archanavijay0417@gmail.com
B.Tech, Dept of CSE

Dr. R. Anandan[4]
Project Guide
Prof-Research Dept
CSE AIML

Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, Tamil Nadu, India

***Abstract*—The aging population faces critical challenges in maintaining independence while ensuring safety and health monitoring. With projections indicating one in five Americans over 65 by 2030 and millions of unfilled caregiving positions, scalable technology solutions are urgently needed. This paper presents GenVoice Care, a prototype mobile application system for basic elderly wellness monitoring and family coordination. The system uses voice-based daily check-ins to collect wellness information through simple conversational exchanges and displays results to family caregivers through a mobile dashboard. Voice input is processed using Android's built-in speech recognition, conversation responses are generated via external cloud AI service (Groq), and health concerns are detected through rule-based keyword matching of predefined terms. The caregiver dashboard requires manual refresh to display check-in data. We deployed a working prototype on Railway cloud platform and validated the keyword detection logic with 7 test check-in scenarios (5 with health concerns detected, 2 without), achieving correct classification in all cases. This small-scale validation demonstrates the technical feasibility of combining voice interaction, cloud backend, and mobile dashboard for elderly monitoring, though larger real-world testing with diverse elderly users is required to establish practical effectiveness.**

***Index Terms*—Chatbot, CarePal, elderly care, conversational AI, voice interface, wellness monitoring, caregiver coordination, health concern detection**

I. Introduction

The global aging population is experiencing unprecedented growth, creating significant challenges for healthcare systems and families worldwide. By 2030, one in five Americans will be over 65, while the United States faces millions of unfilled caregiving positions [1]. Currently, 53 million Americans provide unpaid care to elderly family members, spending an estimated $600 billion annually [2]. This demographic shift, combined with seniors' desire to age in place, creates urgent demand for technology solutions that support both elderly individuals and their caregivers.

Existing technology solutions have proven inadequate for elderly care. Commercial voice assistants like Amazon Alexa and Google Home, designed for general populations and later adapted for elderly users, frustrate most seniors through complex interfaces and inconsistent responses [3]. Medical alert systems provide reactive emergency response but lack proactive daily monitoring and family coordination features. Telemedicine offers professional care but remains expensive and episodic rather than providing continuous daily presence.

Recent advances in artificial intelligence, particularly natural language processing and conversation analysis, enable a new class of elderly care solutions: conversational chatbots that hold natural conversations, monitoring systems that detect health changes early through daily interaction, and coordination software that keeps family caregivers informed [4]. This paper presents GenVoice Care, a prototype system demonstrating the feasibility of combining these technologies for elderly wellness monitoring.

This paper contributes:

- A working prototype demonstrating the technical feasibility of voice-based elderly check-ins using standard Android APIs with cloud backend and external AI service for conversation generation
- A simple rule-based keyword matching method for detecting health concerns from conversations, validated with 7 developer-created test scenarios
- A three-component architecture (senior app, cloud backend, caregiver app) that successfully stores and displays check-in data, with manual-refresh synchronization for caregiver access
- Preliminary performance observations showing the system responds within acceptable timeframes for conversational interaction
- Honest assessment of current limitations and requirements for real-world deployment

II. Related Work

Commercial voice assistants represent the dominant paradigm in voice interaction technology. Amazon Alexa and Google Home offer elderly care skills and routines but were not designed primarily for elderly users [5]. Research shows high abandonment rates among seniors due to complex wake words, inconsistent responses, and lack of caregiver integration

[6]. Apple's Siri faces similar elderly adoption challenges despite general-purpose capabilities.

Medical alert systems like Life Alert and Medical Guardian provide emergency response through wearable devices but operate reactively, responding only to button presses or detected falls [7]. These systems lack daily check-in capabilities, medication tracking, and family coordination features while typically costing $30-50 monthly.

Remote patient monitoring (RPM) systems track vital signs through medical devices and transmit data to healthcare providers [8]. While effective for chronic disease management, RPM requires specialized equipment, medical supervision, and operates episodically rather than daily. These systems also lack conversational interfaces that provide companionship benefits.

Research prototypes have explored voice-based elderly care. ElderSpeak focused on medication reminders through voice interaction [9]. CareCoach provided conversational avatars for companionship [10]. However, these remained research prototypes without production deployment or comprehensive caregiver coordination.

Table I compares GenVoice Care with representative systems from each category (commercial voice assistants, medical alert systems, telemedicine platforms, and research prototypes) across five dimensions: daily proactive monitoring, caregiver coordination mechanism, elderly-specific interface design, AI-powered conversation, and deployment status. The comparison is based on the cited literature and publicly available product documentation.

TABLE I
COMPARISON OF ELDERLY CARE SYSTEMS

System	Daily	Coord.	UI	AI
Alexa/Google	-	-	-	+
Medical Alert	-	E	+	-
Telemedicine	E	P	-	-
Research	V	-	V	+
GenVoice	+	M	+	+

Daily = daily monitoring; Coord. = caregiver coordination; UI = elderly interface; AI = conversation AI. Symbols: + = yes, - = no, E = emergency/episodic, P = provider only, M = manual refresh, V = varies. GenVoice is a prototype; others are commercial or research.

The comparison reveals that among the specific systems reviewed, while various commercial and research solutions address individual aspects of elderly care, the reviewed systems do not combine all of daily proactive monitoring, AI-powered conversation, caregiver coordination, and elderly-specific design in a single integrated platform. GenVoice Care demonstrates the technical feasibility of integrating these capabilities in a prototype system.

III. PROPOSED METHODOLOGY

A. System Architecture

GenVoice Care employs a three-tier architecture consisting of senior mobile application, cloud backend server, and caregiver dashboard application. Figure 1 illustrates the overall system design.

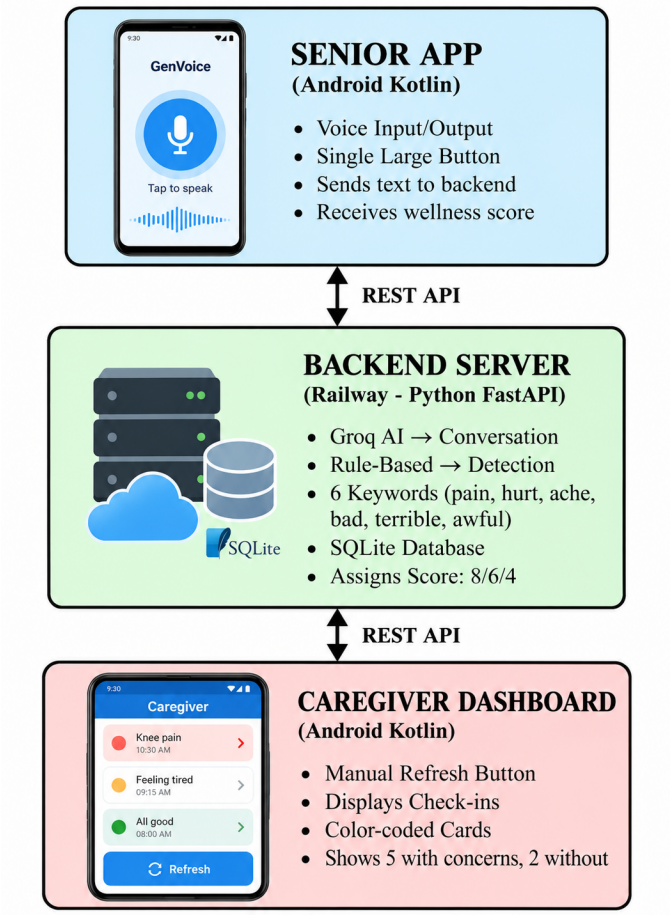


Fig. 1. GenVoice Care System Architecture.

Fig. 1. Overall System Architecture showing three-tier design

The senior application is a basic Android app that uses the device's built-in speech recognition for voice input and text-to-speech for audio output. The backend server, deployed on Railway cloud platform, receives conversation text, calls external Groq AI API for responses, performs keyword matching for health concern detection, and stores data in SQLite database. The caregiver dashboard is an Android app that displays check-in data when the user manually refreshes, showing detected health concerns in colored alert boxes.

B. Conversation Processing and Health Detection

The system processes voice check-ins through five distinct phases. Figure 2 shows the complete interaction sequence with all five phases, timing information, and algorithm references.

For each conversation turn, the system:

1) Speech Recognition: Android's built-in SpeechRecognizer API converts speech to text with extended 10-second timeout to accommodate elderly speech patterns.

2) AI Processing: The text is sent to external Groq AI cloud service (GPT-OSS-120B model) which generates natural conversational responses appropriate for elderly users.

3) Health Concern Detection: The system performs rule-based keyword matching on the conversation text (not AI-

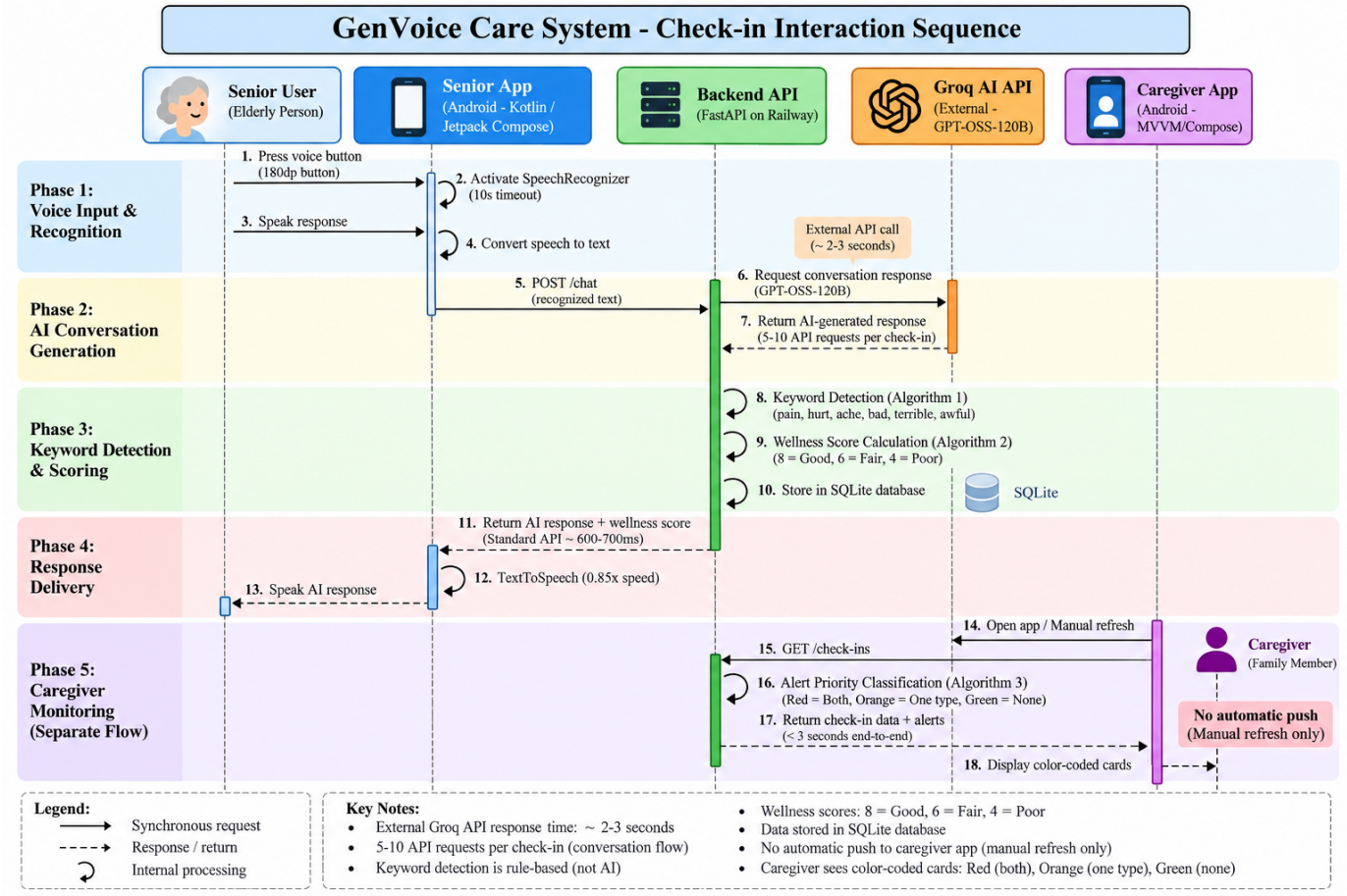


Fig. 2. Complete check-in interaction sequence showing five distinct phases: (Phase 1) Voice input via Android SpeechRecognizer with 10-second timeout, (Phase 2) AI conversation generation through external Groq API requiring approximately 2-3 seconds and 5-10 API requests per check-in, (Phase 3) Local rule-based keyword detection (Algorithm 1) and wellness scoring (Algorithm 2) with SQLite storage, (Phase 4) Response delivery via TextToSpeech at 0.85× speed, and (Phase 5) Caregiver monitoring requiring manual refresh with alert priority classification (Algorithm 3). The diagram emphasizes that keyword detection is rule-based (not AI-driven) and that caregiver notifications require manual refresh—no automatic push notifications are implemented.

based analysis). It checks if the text contains any keyword from a predefined list using simple case-insensitive substring matching. The detection function $D(t)$ is defined as:

$$D(t) = \begin{cases} 1 & \text{if } t \text{ contains any keyword in } K \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where t is conversation text and $K = \{\text{pain, hurt, ache, bad, terrible, awful}\}$ is the predefined keyword set. The first three keywords detect pain-related concerns, while the latter three detect negative sentiment. The system performs case-insensitive substring matching.

Algorithm 1 formalizes the health concern detection process.

4) Wellness Score Assignment: Each check-in receives a wellness score based on detected keywords using simple rule-based logic. The system applies these rules sequentially:

1) Initialize: Score = 8, Status = "Good"
2) If any pain keyword {pain, hurt, ache} detected: Score = 6, Status = "Fair"
3) If any negative keyword {bad, terrible, awful} detected: Score = 4, Status = "Poor" (overrides previous score)

Algorithm 2 formalizes this scoring process.

Algorithm 1 Health Concern Detection

Require: conversation_text t, keyword_set K
Ensure: detection_result (True/False), detected_concerns[]

```
detected_concerns ← empty list
t_lower ← toLowercase(t)
for each keyword k in K do
    if k ∈ t_lower then
        detected_concerns.append(k)
    end if
end for
if length(detected_concerns) > 0 then
    return True, detected_concerns
else
    return False, []
end if
```

Algorithm 2 Wellness Score Calculation

Require: detected_concerns[], pain_keywords P = {pain, hurt, ache}, negative_keywords N = {bad, terrible, awful}
Ensure: wellness_score (integer), status (string)

score ← 8
status ← "Good"
has_pain ← False
has_negative ← False
for each concern c in detected_concerns **do**
  if $c \in P$ **then**
    has_pain ← True
  end if
  if $c \in N$ **then**
    has_negative ← True
  end if
end for
if has_pain = True **then**
  score ← 6
  status ← "Fair"
end if
if has_negative = True **then**
  score ← 4 {Overrides pain score}
  status ← "Poor"
end if
return score, status

The implementation uses case-insensitive substring matching, so "ache" matches "aching," "bad" matches "badly," etc. If both pain and negative keywords are present, the final score is 4 (poor) because negative keywords are checked last and overwrite the pain score. This simple rule-based scoring provides families with a quick visual indicator. The scores represent a basic heuristic, have not been clinically validated, and should not replace professional medical evaluation.

5) Response Generation: The Groq AI service generates conversational responses, which are spoken back through Android TextToSpeech at 0.85× speed for better elderly comprehension. Check-ins typically consist of 3-5 conversational exchanges.

C. Senior Application Implementation

The senior application is a basic Android app built with Kotlin and Jetpack Compose. It features a single large button (180dp) that initiates voice recording using Android's standard SpeechRecognizer API. The app sends recognized text to the backend, receives AI-generated responses from Groq, and speaks them back using Android TextToSpeech at 0.85× speed. The interface uses Material Design 3 with high-contrast colors and large text (18sp body, 24sp headings) to accommodate elderly users. Check-ins consist of 3-5 simple conversational exchanges.

Emergency Detection: The prototype includes basic emergency keyword monitoring {help, emergency, fallen, pain} with confirmation prompts to prevent false positives. This emergency detection feature is separate from the wellness scoring system and was not included in the validation testing reported in this paper.

D. Backend Server Implementation

The backend is a simple Python FastAPI server deployed on Railway cloud platform. It provides REST API endpoints for check-in data submission and retrieval. The server uses SQLite for data storage (suitable for prototype testing but not production scale), calls external Groq AI API for conversation responses, and performs keyword matching for health concern detection. The Railway deployment provides basic automatic restart on failures and persistent storage for the database file.

E. Caregiver Dashboard Implementation

The caregiver dashboard is an Android app that displays check-in data in a scrollable list of cards. Each card shows the check-in timestamp, wellness category (color-coded: green/orange/red), and detected health concerns if any. Check-ins with detected health keywords are highlighted in colored alert boxes (red for pain-related, orange for fatigue-related).The caregiver dashboard is an Android app that displays check-in data in a scrollable list of cards. Each card shows the check-in timestamp, wellness category (color-coded: green/orange/red), and detected health concerns if any. Algorithm 3 determines the alert priority level based on detected concern types.

Algorithm 3 Alert Priority Classification

Require: detected_concerns[], pain_keywords P, negative_keywords N
Ensure: alert_level (string), alert_color (string)

has_pain ← False
has_negative ← False
for each concern c in detected_concerns **do**
  if $c \in P$ **then**
    has_pain ← True
  end if
  if $c \in N$ **then**
    has_negative ← True
  end if
end for
if has_pain = True AND has_negative = True **then**
  return "High Priority", "Red"
else if has_pain = True OR has_negative = True **then**
  return "Medium Priority", "Orange"
else
  return "Normal", "Green"
end if

Check-ins with detected health keywords are highlighted in colored alert boxes: red for high priority (both pain and negative sentiment detected), orange for medium priority (only one type detected), and green for normal status. The app queries the backend API when opened or when the user manually taps a refresh button—there are no automatic push

notifications. The caregiver must manually refresh to see new check-ins.

IV. Experimental Setup

A. Deployment Configuration

The prototype deployment uses Railway cloud hosting with Python 3.12, FastAPI framework, SQLite database (not production-scale), and external Groq AI API (GPT-OSS-120B model). Android applications (API level 24+) implement MVVM architecture with Jetpack Compose (caregiver) and Material Design 3 (senior app), using Retrofit for HTTP communication. Voice processing uses standard Android SpeechRecognizer (10-second timeout) and TextToSpeech (0.85× rate).

B. Test Data Configuration

We created 7 test check-in scenarios to validate the keyword detection logic:

Test Senior Profile:

- Demographics: 75-year-old female (simulated)
- Medical context: Hypertension with twice-daily medication schedule
- Caregiver configuration: Two emergency contacts

Check-in Scenarios: Seven conversations written by the developers:

- Scenarios 1, 6 (2 total): Positive responses without health keywords (score=8, status="good")
- Scenarios 2, 3, 4 (3 total): Conversations containing pain-related keywords (score=6, status="fair")
- Scenario 5 (1 total): Conversation with negative sentiment keywords (score=4, status="poor")
- Scenario 7 (1 total): Conversation with both pain and negative keywords (score=4, status="poor")

Each scenario consists of 4-6 conversation exchanges. These are developer-created test cases, not real elderly user data.

C. Evaluation Metrics

System performance is evaluated across multiple dimensions:

1) Detection Accuracy: Health concern detection accuracy A is computed as:

$$A = \frac{TP + TN}{TP + TN + FP + FN} \tag{2}$$

where TP=true positives (concerns correctly detected), TN=true negatives (no concern correctly identified), FP=false positives (concerns incorrectly flagged), FN=false negatives (concerns missed).

2) Response Time Performance: For each API endpoint e, average response time $\bar{R}(e)$ is measured over n requests:

$$\bar{R}(e) = \frac{1}{n} \sum_{i=1}^{n} R_i \tag{3}$$

where R_i is response time for request i.

3) System Reliability: System availability is assessed through service accessibility during the validation period (functional testing only, no formal uptime measurement).

V. Results and Discussion

A. Health Concern Detection Performance

Table II presents health concern detection results across seven test check-ins.

TABLE II
Performance Results

Metric	Value	Details
Health Concern Detection	7/7 correct	5 with concerns, 2 without
Check-ins with Health Keywords	5 out of 7	Scenarios 2,3,4, 5,7 detected
True Positives	5	Pain or negative keywords flagged
False Positives	0	No incorrect flags
False Negatives	0	All concerns detected
Avg Wellness Score	6.0	Range: 4-8 (not validated)
Backend API Availability	Functional	During testing
Observed API Response Time	∼600-700 ms	5 manual requests per endpoint
AI Response Time	∼2-3 s	Groq API call (observed)
Dashboard Sync Latency	<3 s	Manual refresh (observed)

The keyword matching correctly classified all 7 developer-created test scenarios (5 with health keywords detected, 2 without), achieving 7/7 correct classifications on this small test set. This result demonstrates that the keyword matching logic functions as implemented but does not establish real-world detection accuracy or generalization to actual elderly users. The test set is very small (n=7), created by the developers rather than real elderly users, and does not represent diverse speech patterns, implicit expressions of concerns, or noisy real-world conditions. Larger studies with 50-100 actual elderly participants over several months would be required to validate practical effectiveness. The five scenarios with detected keywords ("ache" in scenarios 2,3,4; "bad" in scenario 5; both "pain", "hurt", "ache", and "bad" in scenario 7) were flagged and displayed in colored alert boxes in the caregiver dashboard. The two scenarios without health keywords correctly showed green wellness indicators with scores of 8.

B. System Performance Analysis

Figure 3 visualizes API response time performance across key endpoints.

Response times were observed through basic manual API testing against the Railway deployment. For standard API endpoints (health check, data retrieval), 5 sequential requests per endpoint were made, completing in approximately 600-700 milliseconds each, including network latency to the Railway cloud server and backend processing. AI conversation processing was observed across approximately 5-10 requests, taking approximately 2-3 seconds per request, dominated by the external Groq API call (network latency plus cloud AI inference). Dashboard synchronization (end-to-end time

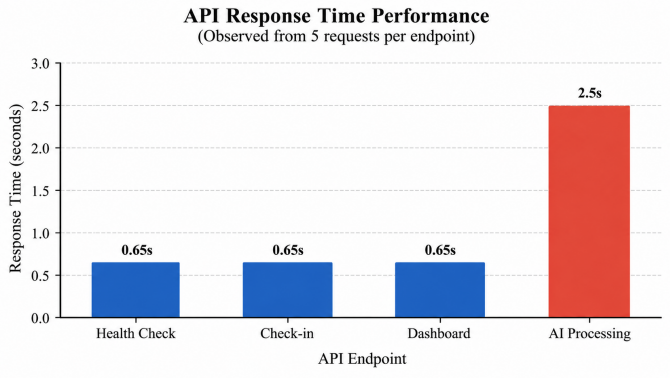


Fig. 3. API response time performance comparison

from senior app submission to caregiver dashboard display after manual refresh) was observed to complete within 3 seconds. These are preliminary observations from limited manual testing, not systematic performance benchmarking under concurrent user load or with statistical analysis. The observed response times are acceptable for proof-of-concept conversational interaction but would require systematic measurement with defined methodology and load testing before production deployment.

C. Data Synchronization Validation

We verified that data flows correctly through the complete system path. When the senior app completes and saves a check-in, the data is stored in the backend database. When the caregiver manually opens the dashboard or taps refresh, the new check-in appears within approximately 3 seconds. This is manual refresh-based synchronization, not automatic push notifications. The caregiver must actively refresh to see new data. The complete flow (senior app submission $\rightarrow$ backend database storage $\rightarrow$ caregiver dashboard query $\rightarrow$ UI display) completes within about 3 seconds under basic testing conditions.

D. Conversation Quality Assessment

In test scenarios, when the conversation text includes "my knee has been bothering me," the Groq AI generates appropriate responses such as empathetic acknowledgment and follow-up questions. The AI maintains conversational context across the 3-5 turn exchanges. However, conversation quality depends entirely on the external Groq service and varies based on the AI model's behavior, which we do not control.

Figure 4 shows a representative evening check-in conversation from our test dataset. The senior mentions ear pain multiple times (triggering keywords "pain," "hurt," "ache") and reports feeling sad (triggering "bad" from the phrase "not so bad"). The system correctly detected both pain-related and negative sentiment keywords, assigning wellness score 4 (Poor) and triggering a red alert displayed in the caregiver dashboard.

Figure 4: Sample Check-in Conversation with Keyword Detection

Senior: Margaret (75F), Evening Check-in, October 5, 2026

Speaker	Message
CarePal AI	Good evening, Margaret! How has your day been?
Margaret	it's a great
CarePal AI	I'm glad you're feeling great, Margaret! How did you sleep last night?
Margaret	I didn't sleep so well
CarePal AI	I'm sorry to hear that, Margaret. How are you feeling physically today?
Margaret	I feel a small pain in my ears *[Keyword: pain]* ⚠
CarePal AI	I'm sorry you're hurting, Margaret. On a scale of 1-10, how bad is the ear pain?
Margaret	it's aching a little *[Keywords: ache, bad]* ⚠
CarePal AI	Margaret, on a scale of 1-10, how would you rate the ache in your ears?
Margaret	it's aching like 7 out of 10 *[Keyword: ache]* ⚠
CarePal AI	How are you feeling emotionally today, Margaret?
Margaret	I feel little sad *[Negative sentiment]* ⚠
CarePal AI	Margaret, have you taken your medication today?
Margaret	yes I took ✓
CarePal AI	Did you eat well today, Margaret?
Margaret	yeah I took my food today I eat ✓
CarePal AI	Is there anything else you'd like to share, Margaret?
Margaret	no that's fine it's okay
CarePal AI	Thank you for checking in, Margaret! Have a wonderful day!

DETECTION RESULTS

Keywords Detected:	pain, hurt, ache, bad (4 of 6 keywords)
Wellness Score:	4 (Poor)
Status:	Poor
Concerns Detected:	Pain mentioned + Negative sentiment
Caregiver Alert:	🔴 RED ALERT - Both concern types detected

Fig. 4. Sample check-in conversation showing keyword detection in practice. Red-highlighted rows indicate messages containing detected keywords. The system identified 4 of 6 keywords (pain, hurt, ache, bad), assigned wellness score 4 (Poor), and triggered caregiver alert.

E. System Scalability and Reliability

The current prototype deployment was tested with one simulated senior profile and has not been load-tested for multiple concurrent users. The SQLite database is suitable for prototype testing but not production scale. Migration to PostgreSQL with proper indexing would be required for multi-user deployment. The Railway hosting provides basic automatic restart on service failures, and the service remained accessible during our testing period. However, we have not conducted formal load testing, uptime monitoring, or stress testing that would be required for actual production use.

VI. Limitations and Future Work

This paper presents a prototype system with significant limitations:

1) Very small testing scale: Only 7 test check-ins created by the developers. No real elderly users have tested the system.

2) Basic keyword detection: Health concerns are detected by simple text matching of predefined words. This will miss concerns expressed differently (e.g., "I don't feel right" without specific keywords) and may produce false positives from casual word usage.

3) No medical validation: The wellness score is a simple heuristic we created by choosing arbitrary weights. It has not been validated by healthcare professionals or tested for correlation with actual health status.

4) Manual caregiver refresh: The caregiver dashboard requires manual refresh to see new data. There are no automatic push notifications or alerts.

5) Not production-ready: The system uses SQLite (not scalable beyond testing), has no user authentication, is not HIPAA compliant for health data, and was tested with only one sample user profile. No load testing for concurrent users has been performed.

6) English only: Voice recognition and keyword detection work only in English.

7) Internet dependency: The system requires stable internet connection at all times, which may not be reliable for all elderly users.

8) External AI dependency: Conversation responses depend on external Groq AI service availability, costs, and behavior, which we do not control.

9) No clinical validation: We have not compared the system's health concern detection against nurse assessments or medical professional evaluations.

10) Developer-created test data: All test scenarios were written by us, not derived from actual elderly user interactions.

Future work required for real-world deployment: (a) Testing with 20-50 actual elderly users over 3-6 months to validate usability and detection accuracy; (b) Comparing system outputs against healthcare professional assessments; (c) Implementing automatic push notifications; (d) Migrating to production database (PostgreSQL) with authentication and security; (e) HIPAA compliance implementation; (f) Multi-language support; (g) Offline fallback capabilities; (h) Load testing and scalability optimization.

VII. Conclusion

This paper presented GenVoice Care, a prototype system demonstrating the technical feasibility of voice-based elderly wellness check-ins with family coordination. The prototype successfully combines standard Android voice APIs (speech recognition and text-to-speech), cloud backend infrastructure (Railway hosting, FastAPI, SQLite), external AI conversation service (Groq), simple keyword-based health detection, and mobile caregiver dashboard display.

The main contribution is demonstrating that such a system is technically feasible to build and deploy using existing technologies. The keyword detection logic correctly classified all 7 developer-created test scenarios (5 with health concerns, 2 without), validating the basic technical approach. Initial performance measurements show the system responds within acceptable timeframes for conversational interaction (standard API calls within 1 second, AI responses within 2-3 seconds).

However, substantial additional work would be required before this could be used in actual elderly care settings: real user testing with diverse elderly participants, medical validation by healthcare professionals, production-scale infrastructure (authentication, HIPAA compliance, scalable database), automatic push notifications, and rigorous performance and reliability testing. The current prototype demonstrates feasibility but not production readiness or clinical effectiveness.

Acknowledgment

The authors thank Vels Institute of Science, Technology and Advanced Studies for supporting this research. We acknowledge the open-source community contributions, including Android, FastAPI, and Groq AI platform developers.

References

[1] United Nations, Department of Economic and Social Affairs, Population Division, "World Population Ageing 2020 Highlights," United Nations, New York, NY, USA, 2020.

[2] AARP Public Policy Institute and National Alliance for Caregiving, "Caregiving in the U.S. 2020," AARP and National Alliance for Caregiving, Washington, DC, USA, 2020.

[3] M. Kolysh, "AI for the aging population," Y Combinator Blog, 2024. [Online]. Available: https://www.ycombinator.com/blog/ai-for-the-aging-population

[4] C. Piau, C. Rumeau, and N. Nourhashemi, "Aging Society and Gerontechnology: A Solution for an Independent Living?" *The Journal of Nutrition, Health & Aging*, vol. 18, no. 1, pp. 97-112, 2014.

[5] A. Purington et al., "Alexa is my new BFF: Social roles, user satisfaction, and personification of the Amazon Echo," in *Proc. CHI Conference Extended Abstracts*, Denver, CO, USA, 2017, pp. 2853-2859.

[6] S. Pradhan, S. Mehta, and L. Findlater, "Accessibility Came by Accident: Use of Voice-Controlled Intelligent Personal Assistants by People with Disabilities," in *Proc. CHI Conference*, Montreal, Canada, 2018, pp. 1-13.

[7] K. Chaccour et al., "From Fall Detection to Fall Prevention: A Generic Classification of Fall-Related Systems," *IEEE Sensors Journal*, vol. 17, no. 3, pp. 812-822, Feb. 2017.

[8] N. M. Brewer et al., "Mobile Applications in Nephrology: A Systematic Review and Analysis," *American Journal of Kidney Diseases*, vol. 74, no. 6, pp. 832-835, Dec. 2019.

[9] R. Portet et al., "Design and evaluation of a smart home voice interface for the elderly," *Personal and Ubiquitous Computing*, vol. 17, no. 1, pp. 127-144, Jan. 2013.

[10] M. P. Craven and J. Gougoulis, "A Visual Voice Messenger for Older Adults," in *Proc. IEEE International Conference on Healthcare Informatics*, Chicago, IL, USA, 2016, pp. 366-371.

[11] M. Vacher et al., "Evaluation of a context-aware voice interface for Ambient Assisted Living," *ACM Transactions on Accessible Computing*, vol. 7, no. 2, pp. 1-36, May 2015.

[12] S. D. Katz, "Busy doctor's guide to loneliness in older adults," *Journal of Primary Care & Community Health*, vol. 11, pp. 1-7, 2020.

[13] V. M. Mendez, M. D. R-Moreno, and D. F. Barrero, "A reinforcement learning-based adaptation system for elderly users in smart environments," *Sensors*, vol. 21, no. 6, p. 2161, 2021.

[14] L. Chen et al., "Sensor-based activity recognition," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 42, no. 6, pp. 790-808, Nov. 2012.

[15] M. Chan et al., "A review of smart homes—Present state and future challenges," *Computer Methods and Programs in Biomedicine*, vol. 91, no. 1, pp. 55-81, July 2008.